

Cahier de Charge

Jeu Multijoueur de Programmation avec Rôle d'Imposteur

Projet Logiciel

8 mars 2026

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Problématique	3
1.3	Objectifs du projet	3
2	Présentation générale du système	3
2.1	Description du concept du jeu	3
2.2	Acteurs du système	4
2.3	Cas d'utilisation principaux	4
3	Fonctionnalités du système	4
3.1	Création et gestion des parties	4
3.2	Attribution aléatoire de l'imposteur	4
3.3	Défis de programmation	4
3.4	Éditeur de code en ligne	5
3.5	Modification du code par l'imposteur	5
3.6	Système de compilation	5
3.7	Système de vote	5
3.8	Assistant intelligent	5
4	Architecture du système	5
4.1	Architecture globale	5
4.2	Communication temps réel	6
4.3	Utilisation du broker	6
4.4	Interaction avec l'API JDoodle	6
5	Conception technique	6
5.1	Modules backend	6
5.2	Gestion des sessions	6
5.3	Gestion des rounds	6
6	Conception du frontend	6
6.1	Interface utilisateur	6
6.2	Éditeur de code	6
6.3	Interface de vote	7
6.4	Interface assistant IA	7
7	Diagrammes recommandés	7

8	Contraintes techniques	7
8.1	Performance	7
8.2	Sécurité	7
8.3	Exécution sécurisée du code	7
8.4	Connexions simultanées	7
9	Planning estimatif	7
10	Perspectives d'amélioration	7
10.1	IA générant des défis	7
10.2	Classement des joueurs	7
10.3	Mode tournoi	8
10.4	Assistant pédagogique avancé	8

1 Introduction

1.1 Contexte

Avec la popularisation des plateformes de programmation collaborative et des jeux multi-joueurs en ligne, de nouvelles formes d'apprentissage interactif apparaissent. Les développeurs et étudiants en informatique cherchent des moyens ludiques et compétitifs d'améliorer leurs compétences en programmation.

Parallèlement, les jeux à rôle caché ont connu un grand succès dans les environnements numériques. L'intégration de ces mécaniques dans un environnement de programmation peut créer une expérience unique mêlant logique, stratégie et collaboration.

Le projet proposé consiste à développer une application web multijoueur dans laquelle des participants résolvent un défi de programmation tout en tentant d'identifier un imposteur infiltré parmi eux.

1.2 Problématique

Les plateformes actuelles de programmation collaborative se concentrent principalement sur la résolution individuelle de problèmes ou sur des compétitions classiques. Elles ne proposent pas de mécanismes de sabotage ou de rôle caché permettant d'introduire une dimension stratégique et sociale.

La problématique principale est donc la suivante :

Comment concevoir une plateforme de programmation collaborative en temps réel intégrant un mécanisme de rôle caché permettant à un joueur de saboter les solutions tout en maintenant une expérience fluide et sécurisée ?

1.3 Objectifs du projet

Les objectifs du projet sont les suivants :

- Concevoir une application web interactive multijoueur.
- Permettre la résolution collaborative de défis de programmation en Java.
- Introduire un mécanisme de rôle caché (imposteur).
- Permettre l'exécution automatique du code via une API externe.
- Assurer une communication temps réel entre les joueurs.
- Intégrer un assistant intelligent capable de fournir des conseils en programmation Java.

2 Présentation générale du système

2.1 Description du concept du jeu

Le système consiste en un jeu multijoueur dans lequel plusieurs joueurs rejoignent une partie. Parmi eux, un joueur est désigné secrètement comme imposteur.

Tous les joueurs reçoivent un défi de programmation en Java. Leur objectif est de produire un code fonctionnel permettant de résoudre le problème donné.

L'imposteur possède une capacité spéciale : il peut modifier secrètement le code d'un autre joueur afin d'introduire des erreurs.

Le système compile et exécute le code via une API externe avec un maximum de trois rounds de compilation.

Les conditions de victoire sont les suivantes :

- Si le code compile correctement au moins une fois : les ingénieurs gagnent.

- Si les trois compilations échouent : l’imposteur gagne.
- Les joueurs peuvent voter pour expulser un joueur suspecté d’être l’imposteur.

2.2 Acteurs du système

Les acteurs principaux sont :

- **Joueur** : participe à une partie et tente de résoudre le défi de programmation.
- **Imposteur** : joueur secret ayant la capacité de saboter le code des autres.
- **Serveur du jeu** : gère les sessions, les communications et les règles du jeu.
- **API externe de compilation** : exécute et compile le code Java.
- **Assistant IA** : analyse le code et fournit des conseils pédagogiques.

2.3 Cas d’utilisation principaux

Les cas d’utilisation principaux incluent :

- Création d’une partie
- Rejoindre une partie
- Attribution du rôle d’imposteur
- Écriture et modification du code
- Sabotage du code par l’imposteur
- Compilation et exécution du code
- Vote pour expulser un joueur
- Consultation de conseils fournis par l’assistant IA
- Affichage du résultat de la partie

3 Fonctionnalités du système

3.1 Création et gestion des parties

Le système doit permettre :

- la création de nouvelles parties
- la gestion des joueurs connectés
- la gestion du nombre maximal de joueurs
- le démarrage de la partie lorsque tous les joueurs sont prêts

3.2 Attribution aléatoire de l’imposteur

Lors du lancement d’une partie, le serveur attribue aléatoirement le rôle d’imposteur à un joueur.

Cette information reste secrète et n’est visible que par le joueur concerné.

3.3 Défis de programmation

Le système doit afficher un problème de programmation en Java contenant :

- une description du problème
- un exemple d’entrée
- un exemple de sortie

3.4 Éditeur de code en ligne

Les joueurs disposent d'un éditeur de code intégré permettant :

- l'écriture de code Java
- la modification du code
- l'envoi du code pour compilation

3.5 Modification du code par l'imposteur

L'imposteur possède la capacité de modifier le code d'un joueur cible afin d'introduire des erreurs de compilation ou de logique.

Ces modifications doivent être invisibles pour les autres joueurs.

3.6 Système de compilation

Le code est compilé via l'API externe.

Le système permet un maximum de trois rounds de compilation.

Chaque round produit :

- un résultat de compilation
- un résultat d'exécution
- des messages d'erreur éventuels

3.7 Système de vote

Les joueurs peuvent lancer un vote afin d'expulser un joueur suspecté d'être l'imposteur.

Si l'imposteur est expulsé :

- les ingénieurs gagnent la partie

3.8 Assistant intelligent

Un assistant basé sur l'intelligence artificielle pourra analyser le code soumis par les joueurs et fournir des conseils.

Cet assistant proposera :

- des explications sur les erreurs de compilation
- des conseils sur la syntaxe Java
- des suggestions d'amélioration du code
- des rappels de concepts Java

Il ne donnera pas la solution directe au problème afin de préserver l'équilibre du jeu.

4 Architecture du système

4.1 Architecture globale

Le système suit une architecture client-serveur composée de :

- Frontend : React.js
- Backend : Spring Boot
- API externe : JDoodle

4.2 Communication temps réel

La communication entre les clients et le serveur est réalisée via WebSockets afin d'assurer une synchronisation en temps réel des actions des joueurs.

4.3 Utilisation du broker

Deux options sont envisagées :

- Simple Broker intégré dans Spring Boot
- Apache Kafka pour une architecture plus scalable

4.4 Interaction avec l'API JDoodle

Le backend envoie le code Java à l'API JDoodle qui retourne :

- le résultat de compilation
- la sortie du programme
- les messages d'erreur

5 Conception technique

5.1 Modules backend

- gestion des parties
- gestion des joueurs
- gestion WebSocket
- gestion de la compilation
- gestion des rounds
- intégration de l'assistant IA

5.2 Gestion des sessions

Chaque partie possède :

- un identifiant unique
- une liste de joueurs
- un imposteur
- un état de jeu

5.3 Gestion des rounds

Le système gère trois rounds maximum de compilation.

6 Conception du frontend

6.1 Interface utilisateur

Interface simple permettant de suivre l'état du jeu.

6.2 Éditeur de code

Éditeur avec coloration syntaxique pour Java.

6.3 Interface de vote

Permet de voter pour expulser un joueur suspect.

6.4 Interface assistant IA

Zone permettant d'afficher les conseils générés par l'assistant.

7 Diagrammes recommandés

- diagramme de cas d'utilisation
- diagramme de séquence
- diagramme d'architecture

8 Contraintes techniques

8.1 Performance

Le système doit supporter plusieurs parties simultanées.

8.2 Sécurité

Validation des actions côté serveur.

8.3 Exécution sécurisée du code

Le code est exécuté dans un environnement externe isolé.

8.4 Connexions simultanées

Gestion de multiples connexions WebSocket.

9 Planning estimatif

- Analyse : 1 semaine
- Backend : 2 semaines
- Frontend : 2 semaines
- Tests : 1 semaine
- Documentation : 1 semaine

10 Perspectives d'amélioration

10.1 IA générant des défis

Un module d'intelligence artificielle pourrait générer automatiquement de nouveaux défis de programmation.

10.2 Classement des joueurs

Ajout d'un système de score et de classement.

10.3 Mode tournoi

Organisation de compétitions entre plusieurs parties.

10.4 Assistant pédagogique avancé

Amélioration de l'assistant IA pour proposer des analyses plus poussées du code et des explications pédagogiques détaillées.